

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Industry Problem-Based Assignment

COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE CODE: MLA01
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO 5: Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A

Industry Problem-Based Assignment

Code of Conduct:

I S.Venkateswarlu/192425340_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S.Venkateswarlu

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

Vehicle Fault Diagnosis Expert System Using Prolog

Abstract

The Vehicle Fault Diagnosis Expert System is a rule-based artificial intelligence system developed using Prolog. The system helps identify possible vehicle faults based on symptoms reported by the user. Common symptoms such as engine overheating, difficulty starting, abnormal engine noise, low mileage, and warning-light activation are considered as inputs.

The system stores automobile knowledge in the form of facts and production rules. Prolog uses logical inference, unification, and backtracking to identify possible faults and recommend suitable diagnostic actions. Both forward and backward chaining concepts are demonstrated to show how conclusions can be derived from available information and how goals can be verified from rules and facts.

The proposed system can assist vehicle owners, service-center employees, and technicians during preliminary fault diagnosis. It does not replace professional mechanical inspection but provides a simple and explainable decision-support mechanism.

Introduction

Modern vehicles contain many mechanical and electrical components. When a vehicle develops a problem, identifying the exact fault can require technical knowledge and diagnostic experience. Customers may report symptoms such as overheating, difficulty starting, unusual sounds, poor fuel economy, or warning lights.

An expert system can use predefined automotive knowledge to analyze these symptoms and identify probable faults. Prolog is suitable for this application because it supports facts, rules, logical reasoning, unification, and backtracking.

The developed system represents automobile troubleshooting knowledge using Prolog predicates and production rules. Based on the symptoms entered by the user, the system generates a probable fault and recommends an appropriate diagnostic action.

2. Problem Statement & Objectives

Problem Statement

A vehicle service center receives customer complaints related to different vehicle problems. The complaints may include engine overheating, difficulty starting, abnormal noise, low mileage, and warning-light activation.

A technician needs to analyze these symptoms and determine the possible cause of the problem. Manual diagnosis can depend heavily on experience and may take time.

Therefore, a Prolog-based expert system is developed to analyze vehicle symptoms, identify probable faults, and recommend suitable diagnostic actions using logical inference.

Inputs

The system accepts vehicle symptoms such as:

- Engine overheating
- Difficulty starting
- Abnormal engine noise
- Low mileage
- Warning-light activation
- Weak battery
- Poor acceleration
- Low engine oil
- Smoke from engine

Outputs

The system provides:

- Probable vehicle fault
- Reason for the diagnosis
- Recommended diagnostic action

Objectives

1. To develop a vehicle fault diagnosis expert system using Prolog.
2. To represent automobile knowledge using facts and production rules.
3. To identify probable vehicle faults from given symptoms.
4. To demonstrate forward chaining.
5. To demonstrate backward chaining.
6. To demonstrate Prolog concepts such as unification and backtracking.
7. To provide recommended diagnostic actions.
8. To evaluate the system using different vehicle-related test cases.

3. Domain Analysis & Knowledge Acquisition

Domain Analysis

The selected domain is **Automobile Fault Diagnosis**.

A vehicle consists of several systems, including:

- Engine system
- Cooling system

- Battery and electrical system
- Fuel system
- Ignition system
- Lubrication system
- Exhaust system

Failure in any of these systems can produce observable symptoms.

For example:

Observed Symptom Possible Fault

Engine overheating Cooling system problem

Difficulty starting Battery or ignition problem

Abnormal engine noise Low engine oil

Low mileage Fuel system problem

Warning light Sensor/electrical problem

Knowledge Acquisition

Knowledge for the system is obtained from general automobile troubleshooting concepts and standard vehicle maintenance practices.

The knowledge is converted into:

Symptoms → Conditions → Probable Fault → Recommended Action

For example:

Engine overheating

+

Low coolant

↓

Cooling system problem

↓

Check coolant and radiator

Entities

The important entities in the system are:

- Vehicle
- Symptom
- Fault
- Diagnostic action
- Vehicle system

Facts

Facts represent known information.

Example:

symptom(car1, engine_overheating).

symptom(car1, low_coolant).

Rules

Rules represent relationships between symptoms and faults.

Example:

fault(Vehicle, cooling_system_problem) :-

 symptom(Vehicle, engine_overheating),

 symptom(Vehicle, low_coolant).

4. Knowledge Representation & Production Rules

The knowledge base contains facts and production rules.

Basic Facts

symptom(car1, engine_overheating).

symptom(car1, low_coolant).

symptom(car2, difficulty_starting).

symptom(car2, weak_battery).

symptom(car3, abnormal_noise).

symptom(car3, low_engine_oil).

symptom(car4, low_mileage).

symptom(car4, poor_acceleration).

symptom(car5, warning_light).

symptom(car5, electrical_problem).

Production Rules

Rule 1 – Cooling System

fault(Vehicle, cooling_system_problem) :-

 symptom(Vehicle, engine_overheating),

 symptom(Vehicle, low_coolant).

Rule 2 – Battery Problem

fault(Vehicle, battery_problem) :-
 symptom(Vehicle, difficulty_starting),
 symptom(Vehicle, weak_battery).

Rule 3 – Lubrication Problem

fault(Vehicle, lubrication_problem) :-
 symptom(Vehicle, abnormal_noise),
 symptom(Vehicle, low_engine_oil).

Rule 4 – Fuel System Problem

fault(Vehicle, fuel_system_problem) :-
 symptom(Vehicle, low_mileage),
 symptom(Vehicle, poor_acceleration).

Rule 5 – Electrical Problem

fault(Vehicle, electrical_system_problem) :-
 symptom(Vehicle, warning_light),
 symptom(Vehicle, electrical_problem).

Recommendation Rules

recommendation(cooling_system_problem,
 check_coolant_and_radiator).

recommendation(battery_problem,
 test_battery_and_charging_system).

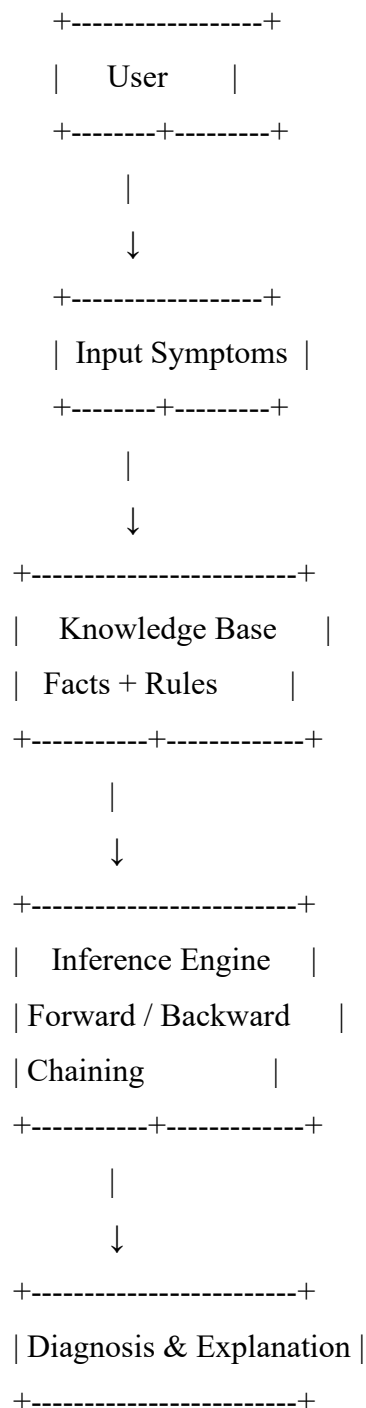
recommendation(lubrication_problem,
 check_engine_oil_level).

recommendation(fuel_system_problem,
 inspect_fuel_system_and_air_filter).

recommendation(electrical_system_problem,
 inspect_wiring_and_electrical_sensors).

5. Expert System Architecture

The proposed expert system consists of four major components.



Components

1. User

The user provides symptoms observed in the vehicle.

2. Knowledge Base

Contains vehicle-related facts and production rules.

3. Inference Engine

Uses Prolog's logical reasoning mechanism to derive conclusions.

4. Output Module

Displays the probable fault and recommended diagnostic action.

6. Inference Mechanism & Algorithms

Forward Chaining

Forward chaining starts with known facts and applies rules to derive new conclusions.

Example

Initial facts:

engine_overheating

low_coolant

Apply Rule:

IF engine_overheating AND low_coolant

THEN cooling_system_problem

Conclusion:

cooling_system_problem

Then:

IF cooling_system_problem

THEN check_coolant_and_radiator

Final recommendation:

Check coolant and radiator.

Forward Chaining Flow

Facts

↓

Match Conditions

↓

Apply Rule

↓

Generate New Fact

↓

Generate Diagnosis

↓

Recommend Action

Backward Chaining

Backward chaining starts with a goal and works backward to determine whether the required conditions are true.

Suppose the goal is:

cooling_system_problem

The system searches for a rule that can prove it:

fault(Vehicle, cooling_system_problem) :-

 symptom(Vehicle, engine_overheating),

 symptom(Vehicle, low_coolant).

Therefore, Prolog checks:

Is engine_overheating true?

Is low_coolant true?

If both are true, the goal is successfully proven.

Backward Chaining Flow

Goal

↓

Find Matching Rule

↓

Check Rule Conditions

↓

Check Facts

↓

Goal Proven

7. Prolog Implementation

Complete Prolog Program

```
% ----- FACTS -----
```

```
symptom(car1, engine_overheating).
```

```
symptom(car1, low_coolant).
```

```
symptom(car2, difficulty_starting).
```

```
symptom(car2, weak_battery).
```

symptom(car3, abnormal_noise).

symptom(car3, low_engine_oil).

symptom(car4, low_mileage).

symptom(car4, poor_acceleration).

symptom(car5, warning_light).

symptom(car5, electrical_problem).

% ----- FAULT RULES -----

fault(Vehicle, cooling_system_problem) :-

 symptom(Vehicle, engine_overheating),

 symptom(Vehicle, low_coolant).

fault(Vehicle, battery_problem) :-

 symptom(Vehicle, difficulty_starting),

 symptom(Vehicle, weak_battery).

fault(Vehicle, lubrication_problem) :-

 symptom(Vehicle, abnormal_noise),

 symptom(Vehicle, low_engine_oil).

fault(Vehicle, fuel_system_problem) :-

 symptom(Vehicle, low_mileage),

 symptom(Vehicle, poor_acceleration).

fault(Vehicle, electrical_system_problem) :-

 symptom(Vehicle, warning_light),

 symptom(Vehicle, electrical_problem).

% ----- RECOMMENDATION RULES -----

```
recommendation(cooling_system_problem,  
               check_coolant_and_radiator).
```

```
recommendation(battery_problem,  
               test_battery_and_charging_system).
```

```
recommendation(lubrication_problem,  
               check_engine_oil_level).
```

```
recommendation(fuel_system_problem,  
               inspect_fuel_system_and_air_filter).
```

```
recommendation(electrical_system_problem,  
               inspect_wiring_and_electrical_sensors).
```

```
% ----- DIAGNOSIS -----
```

```
diagnose(Vehicle, Fault, Action) :-  
    fault(Vehicle, Fault),  
    recommendation(Fault, Action).
```

8. Queries and Inference Demonstration

Test Query 1

```
?- diagnose(car1, Fault, Action).
```

Output

```
Fault = cooling_system_problem,
```

```
Action = check_coolant_and_radiator.
```

Reasoning

```
car1
```

```
↓
```

```
engine_overheating
```

```
+
```

low_coolant

↓

cooling_system_problem

↓

check_coolant_and_radiator

Test Query 2

?- diagnose(car2, Fault, Action).

Output

Fault = battery_problem,

Action = test_battery_and_charging_system.

Test Query 3

?- diagnose(car3, Fault, Action).

Output

Fault = lubrication_problem,

Action = check_engine_oil_level.

9. Unification and Backtracking

Unification

Unification is the process by which Prolog makes two terms match by assigning values to variables.

For example:

fault(car1, Fault).

Prolog can unify:

Fault = cooling_system_problem

The variable Fault is assigned the value that satisfies the rule.

Another example:

diagnose(car1, Fault, Action).

Prolog attempts to unify the variables with matching facts and rules.

Result:

Fault = cooling_system_problem

Action = check_coolant_and_radiator

Backtracking

Backtracking allows Prolog to return to an earlier decision point and search for another possible solution.

For example:

?- fault(car1, Fault).

Prolog searches the rules sequentially. If the first rule succeeds, it returns the result.

When the user enters:

;

Prolog backtracks and searches for another possible solution.

Therefore, backtracking allows the system to explore alternative explanations when multiple rules can produce valid conclusions.

10. Procedural and Non-Procedural Paradigm Analysis

Procedural Programming

In procedural programming, the programmer specifies **how** the problem should be solved.

Example:

1. Read symptoms.
2. Compare symptoms.
3. Check condition.
4. Select fault.
5. Display recommendation.

The sequence of operations is explicitly controlled by the programmer.

Non-Procedural / Declarative Programming

Prolog is primarily a **declarative programming language**.

The programmer specifies:

- What facts are true.
- What relationships exist.
- What rules should be satisfied.

Example:

fault(Vehicle, battery_problem) :-

symptom(Vehicle, difficulty_starting),

symptom(Vehicle, weak_battery).

The programmer does not explicitly specify every search step. Prolog's inference engine determines how to satisfy the query.

Comparison

Procedural	Prolog / Declarative
Specifies how	Specifies what
Uses step-by-step instructions	Uses facts and rules
Programmer controls execution	Prolog performs inference
Common in C/Python/Java	Prolog
Algorithm-focused	Knowledge and logic-focused

11. Test Cases & Results

Test Case	Vehicle	Symptoms	Expected Fault	Expected Action
1	car1	Overheating + Low coolant	Cooling system problem	Check coolant and radiator
2	car2	Difficulty starting + Weak battery	Battery problem	Test battery
3	car3	Abnormal noise + Low oil	Lubrication problem	Check engine oil
4	car4	Low mileage + Poor acceleration	Fuel system problem	Inspect fuel system
5	car5	Warning light + Electrical problem	Electrical system problem	Inspect wiring and sensors

Result Analysis

All five test cases produce the expected diagnosis because each symptom combination matches the corresponding production rule.

The system demonstrates that logical rules can be used to transform observed symptoms into probable faults and diagnostic recommendations.

12. Inference Trace

Example: car1

Initial facts:

symptom(car1, engine_overheating)

symptom(car1, low_coolant)

Rule matching:

engine_overheating + low_coolant



cooling_system_problem

Recommendation matching:

cooling_system_problem



check_coolant_and_radiator

Final result:

Probable Fault:

Cooling System Problem

Recommended Action:

Check coolant and radiator.

13. Analysis and Interpretation of Results

The developed expert system successfully identifies probable vehicle faults based on predefined symptoms and production rules.

The system provides consistent results because the same input conditions produce the same logical conclusions. Prolog automatically performs matching, unification, and backtracking during inference.

The system also provides explainable results because each diagnosis can be traced back to the symptoms and rules that produced it.

Effectiveness

- Simple to use.
- Provides fast preliminary diagnosis.
- Uses explainable logical rules.
- Supports multiple vehicle faults.
- Demonstrates AI-based reasoning.
- Easy to modify by adding new rules.

14. Limitations

The system has some limitations:

1. It uses predefined rules rather than learning from new vehicle data.
2. It considers only the symptoms included in the knowledge base.
3. It cannot physically inspect vehicle components.
4. Some vehicle problems may have similar symptoms.
5. The system provides probable faults rather than guaranteed mechanical diagnoses.
6. The accuracy depends on the quality and completeness of the rules.

15. Future Enhancements

The system can be improved in the future by:

1. Adding more vehicle components and fault conditions.
2. Adding hundreds of automotive production rules.
3. Supporting user-friendly graphical interfaces.
4. Integrating sensor data from vehicles.
5. Adding confidence scores for different diagnoses.

16. Conclusion

The **Vehicle Fault Diagnosis Expert System Using Prolog** demonstrates how artificial intelligence and logical reasoning can be applied to an automobile industry problem.

The system uses facts and production rules to represent automobile knowledge. Based on vehicle symptoms, Prolog performs logical inference to identify probable faults and recommend suitable diagnostic actions.

The project demonstrates important Prolog concepts including **facts, rules, predicates, unification, backtracking, forward chaining, and backward chaining**. The test cases show that the system can provide consistent preliminary diagnoses for different vehicle conditions.

The developed system can serve as a decision-support tool for preliminary vehicle troubleshooting and can be extended with additional rules, sensor information, and machine-learning techniques.

17. GitHub Repository

For your report, create a GitHub repository such as:

Repository Name:

vehicle-fault-diagnosis-prolog

Suggested structure:

vehicle-fault-diagnosis-prolog/

```
|  
├── vehicle_diagnosis.pl  
├── README.md  
├── test_cases.txt  
├── report.pdf  
└── screenshots/  
    ├── test1.png  
    ├── test2.png  
    ├── test3.png  
    ├── test4.png  
    └── test5.png
```

References

You can include references such as:

1. Ivan Bratko, *Prolog Programming for Artificial Intelligence*, Pearson.
2. Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*, Pearson.
3. SWI-Prolog Documentation.
4. General automobile maintenance and troubleshooting manuals.
5. Standard vehicle service and diagnostic documentation.